Análise de Resiliência e Propagação de Vulnerabilidades em Grafos de Dependências do Ecossistema Python

Márcio Levi Sales Prado — RA 183680
MC859 — UNICAMP, Abril de 2026

1. Introdução

Este projeto modela o ecossistema de pacotes Python (PyPI) como um **grafo direcionado**, em que cada vértice representa um pacote de software e cada aresta A → B indica que o pacote A declara dependência direta de runtime sobre B. O objetivo central é identificar pacotes de alto risco — aqueles cuja falha de segurança pode se propagar amplamente pelo ecossistema — cruzando métricas estruturais do grafo (centralidade, grau, componentes fortemente conexas) com vulnerabilidades conhecidas provenientes do OSV (Open Source Vulnerabilities) e do GitHub Advisory Database.

Coleta de dados. A lista dos 3.000 pacotes mais baixados nos últimos 30 dias foi obtida do serviço público *top-pypi-packages* (hugovk.github.io). Para cada pacote-semente, a API JSON do PyPI (https://pypi.org/pypi/{package}/json) foi consultada para extrair suas dependências diretas de runtime. O processo expandiu o conjunto de nós recursivamente até atingir um limite de 8.000 vértices, totalizando 3.359 pacotes únicos e 9.659 arestas de dependência. Todos os dados são públicos e não requerem anonimização.

Repositório público com os grafos e scripts: https://github.com/marcinholsp/Projeto_MC859

2. Tamanho do Grafo

Métrica	Valor
Número de vértices	3.359
Número de arestas	9.659
Grau médio (in + out)	5.75
In-degree médio	2.88
Out-degree médio	2.88
In-degree máximo	495 (typing-extensions)
Out-degree máximo	162

O grafo exibe distribuição de graus em *lei de potência* (scale-free), com expoentes –1,13 para o in-degree e –1,44 para o grau total (estimados por regressão log-log). Hubs como **typing-extensions** (in-degree 495), **requests** (253) e **numpy** (215) concentram a maior parte das dependências.

3. Distribuição de Graus

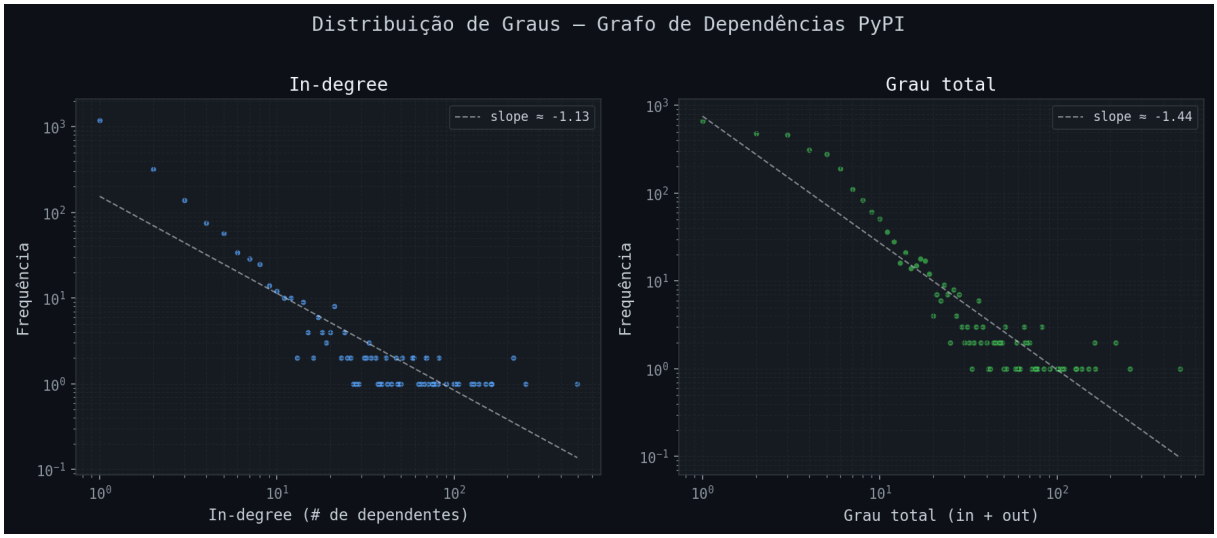

Figura 1. Distribuição de graus em escala log-log. À esquerda: in-degree (número de dependentes); à direita: grau total (in + out). A linha tracejada indica o ajuste por lei de potência.

4. Componentes Fortemente Conexas (CFCs)

O algoritmo de Kosaraju identificou **3.351** componentes fortemente conexas (CFCs). Desse total. **3.349 são singletons** (99.94 %). confirmando que o ecossistema PyPI é essencialmente *acíclico* — dependências circulares são raras e restritas a grupos muito pequenos. Apenas 2 CFCs possuem mais de um vértice: uma com 2 vértices e outra com **8 vértices** (a maior CFC encontrada).

5. Distribuição dos Tamanhos das CFCs

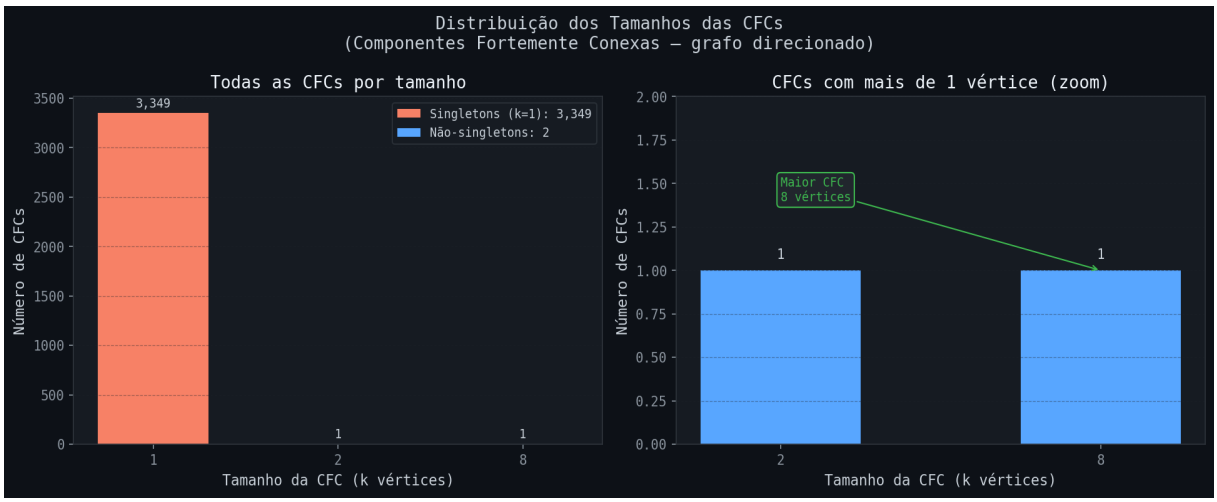

Figura 2. Distribuição dos tamanhos das CFCs. À esquerda: visão geral (3.349 singletons dominam o histograma). À direita: zoom nas CFCs com mais de 1 vértice — apenas duas existem, de tamanhos 2 e 8 vértices, evidenciando a quase-ausência de ciclos de dependência.

A predominância de singletons é esperada em ecossistemas de pacotes maduros: gerentes de pacotes como *pip* exigem grafo acíclico para resolver dependências deterministicamente. As raras dependências circulares encontradas podem indicar versões de compatibilidade cruzada ou interdependências opcionais não declaradas corretamente.